

# Apuntes – Clase 8

## **Traducción de direcciones**

### **Sistemas Operativos**

Universidad de Antioquia

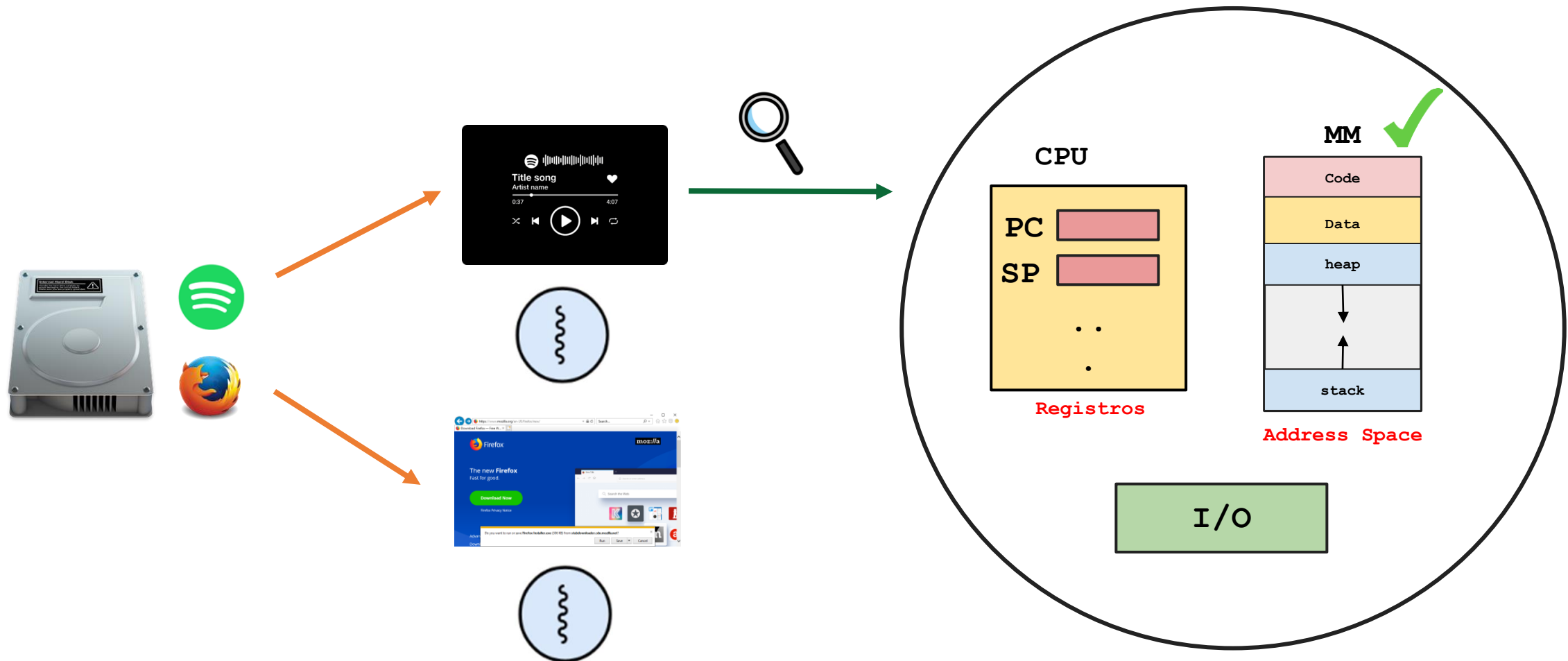
Facultad Ingeniería

Ingeniería de Sistemas

# Proceso

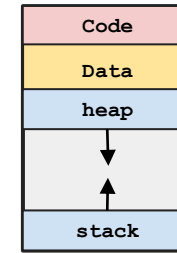
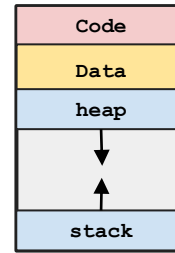
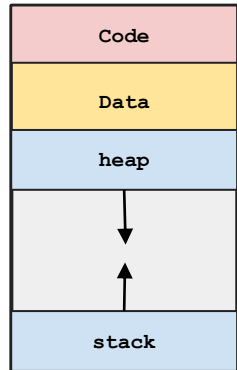
Un proceso es un **programa en ejecución**.

Proceso = CPU + **Memory** + I/O Info



# Virtualización de Memoria - Contextualización

Virtualización de memoria: Cada proceso cree que **tiene su propia memoria**.



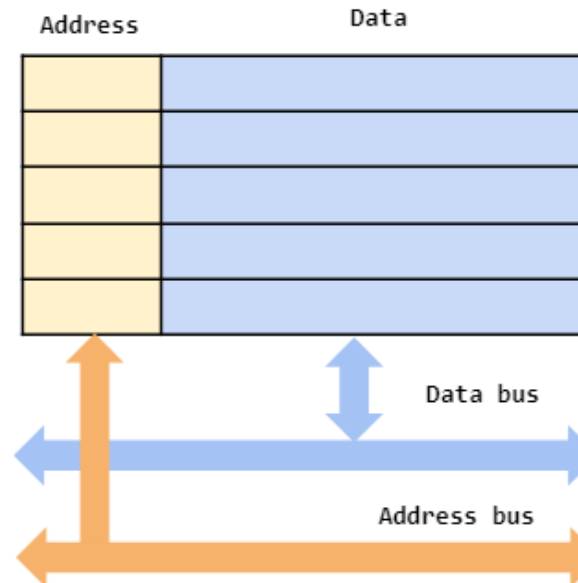
Ilusión



P0



P1

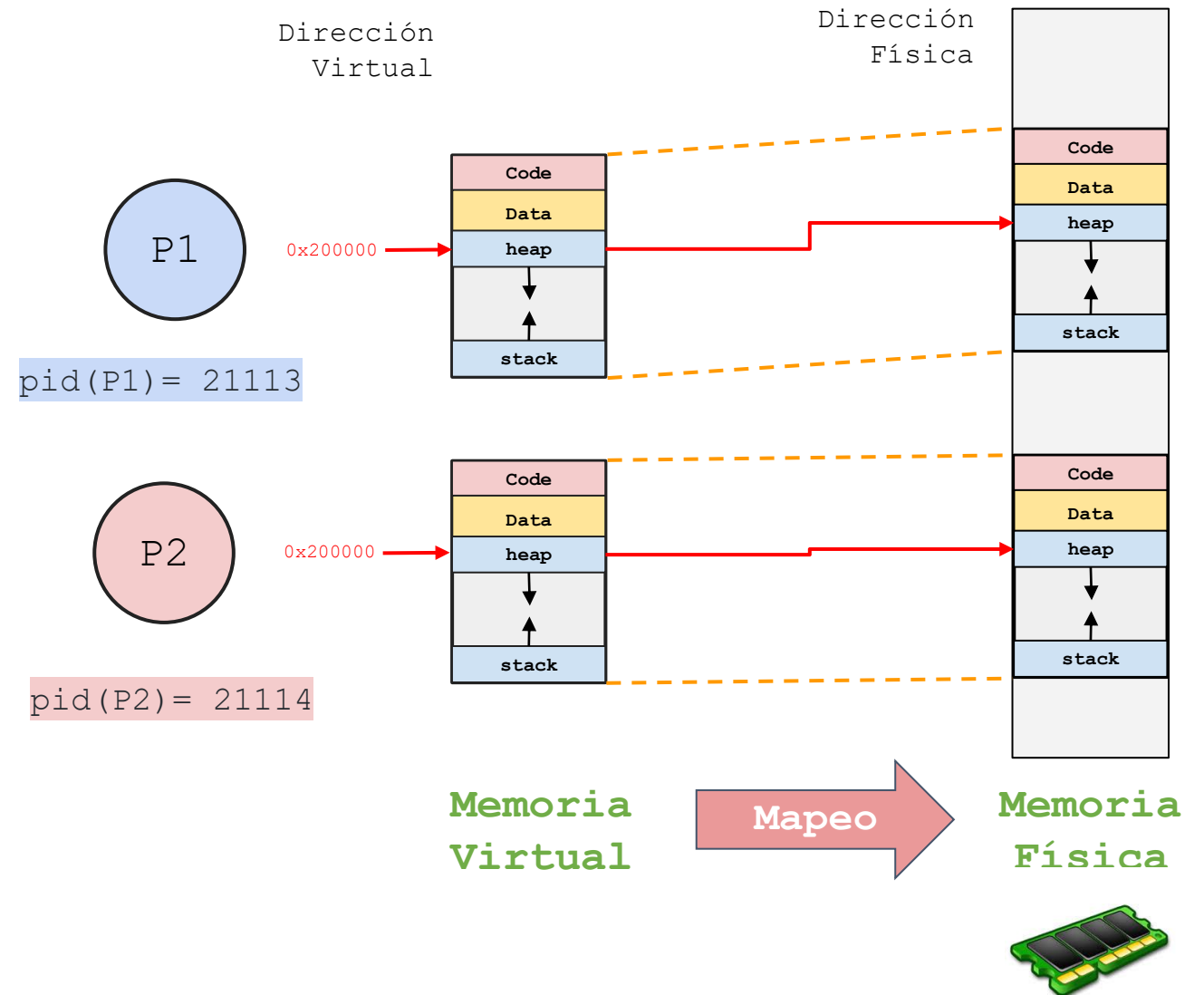


Recurso

# Virtualización de Memoria - Contextualización

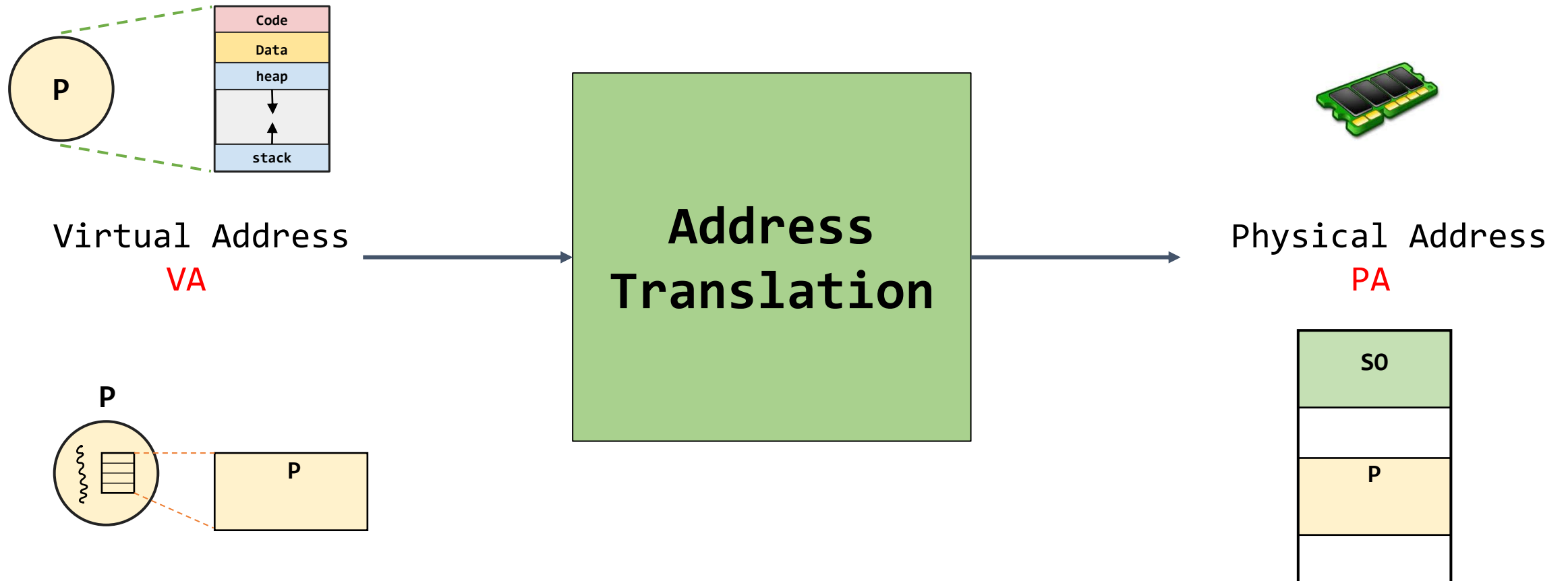
**Virtualización de memoria:**  
Cada proceso cree que **tiene su propia memoria**.

```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```



# Hardware Address Translation - Address Translation (AT)

Mecanismo mediante la traducción de **direcciones virtuales** a **físicas**.



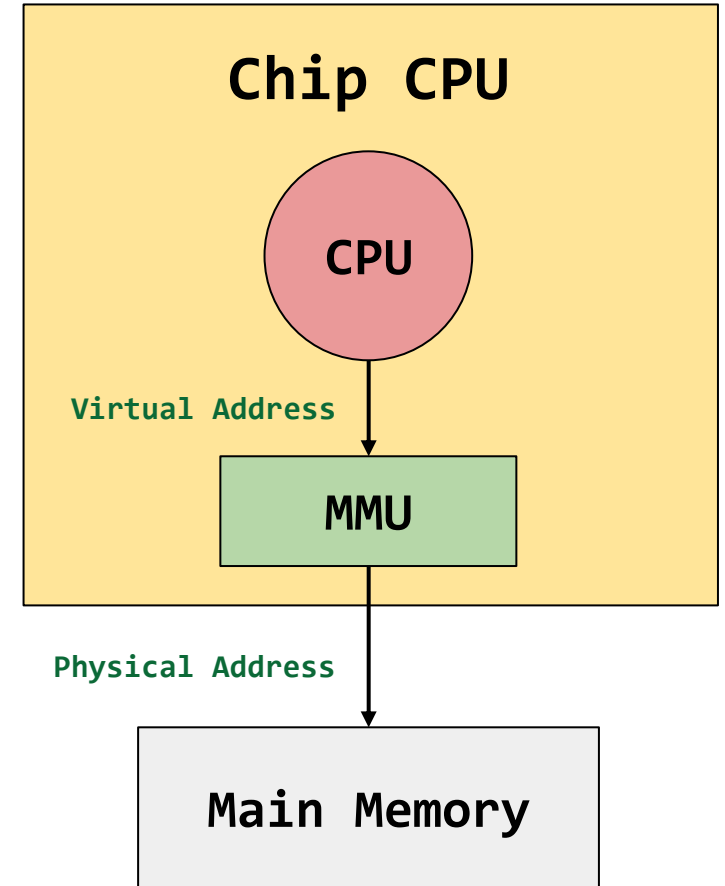
# Contextualización

## Conceptos claves

- La Virtualización de memoria **requiere soporte en hardware**:
  - Similar al mecanismo de **Ejecución Directa Limitada (Limited Direct Execution)**.
  - **Objetivos**:
    - **Eficiencia**: Uso del recurso lo mejor posible.
    - **Control**: Protección y acceso correcto a memoria.

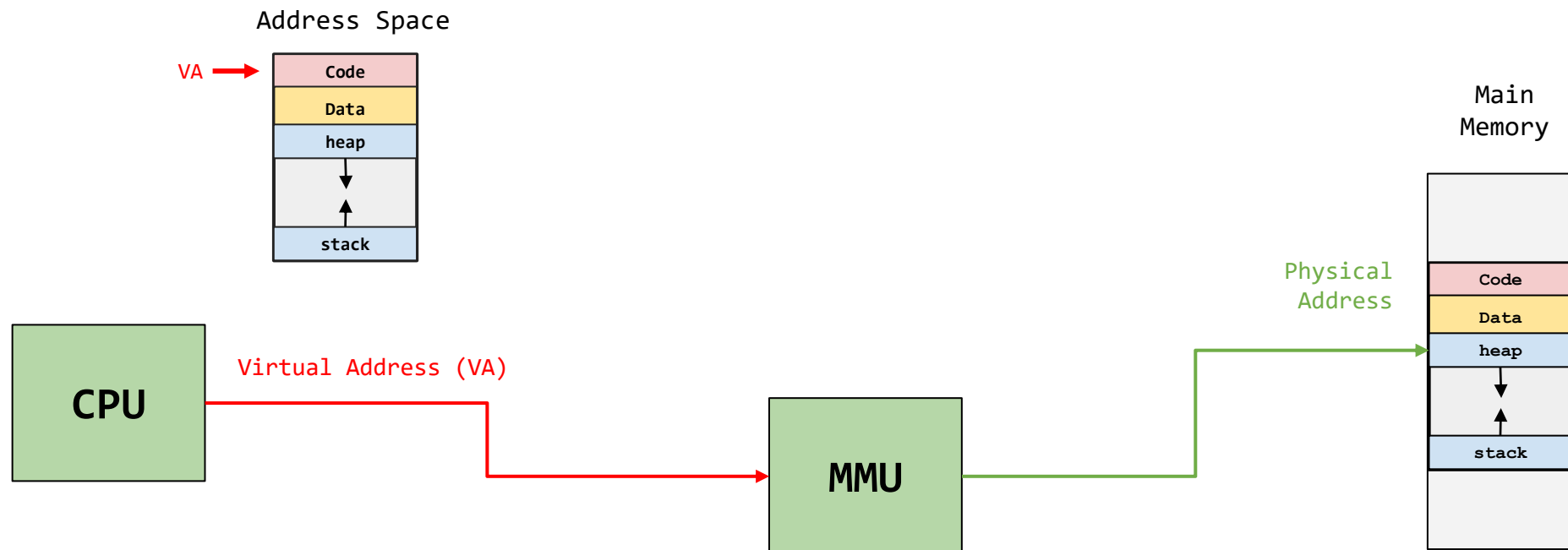
### Soporte en hardware

- Modos de CPU
- Interrupciones
- Registros
- Segment tables
- TLB
- Page Tables
- ...



# Traducción de direcciones

- Las direcciones del espacio de direcciones son **virtuales**.
- El hardware traduce las **direcciones virtuales** a **direcciones físicas**:
  - La información es **almacenada** en las direcciones físicas.
- El SO debe configurar el hardware en ciertos momentos clave.
  - El SO administra la memoria e interviene cuando es necesario.



# Contextualización

## Ejemplo

Para el siguiente fragmento de código asuma que la variable `x` está en la dirección 15360 (15K) y que las direcciones de las instrucciones generadas en lenguaje ensamblador (similar al MIPS) están en las direcciones mostradas a continuación.

### Lenguaje C

```
void func() {  
    int x = 3000;  
    x = x + 3;  
}
```

### Ensamblador

```
128 lw $S0, 15360  
132 addi $S0, $S0, 3  
135 sw $S0, 15360
```

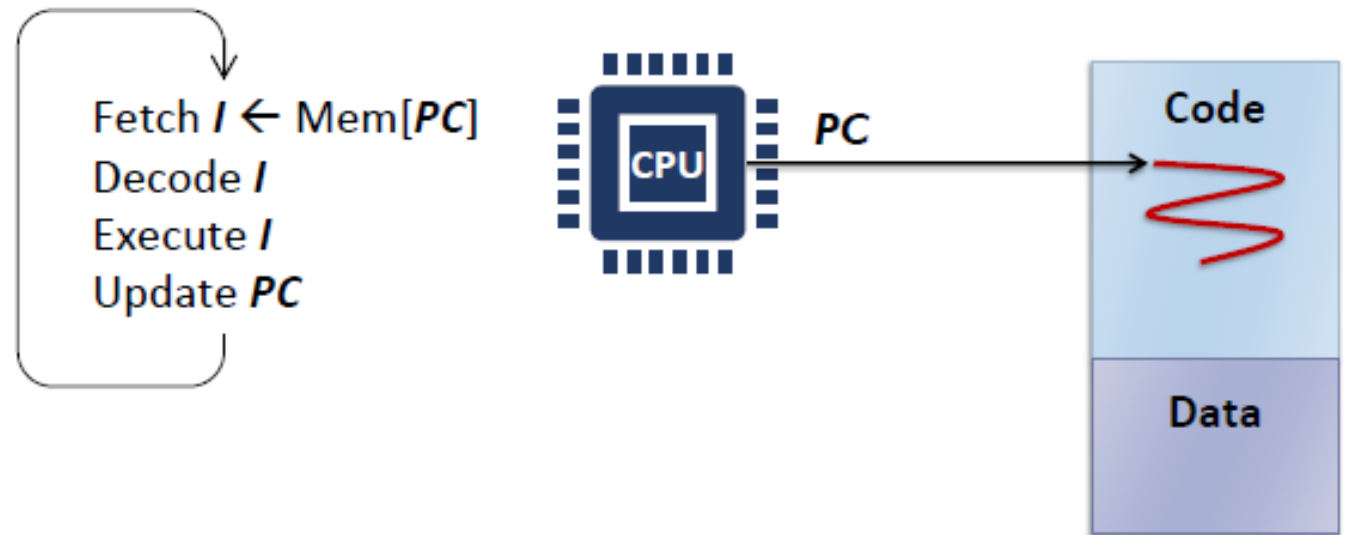
¿Cómo se ejecutaría el código asociado a la instrucción `x = x + 3`?

# Contextualización

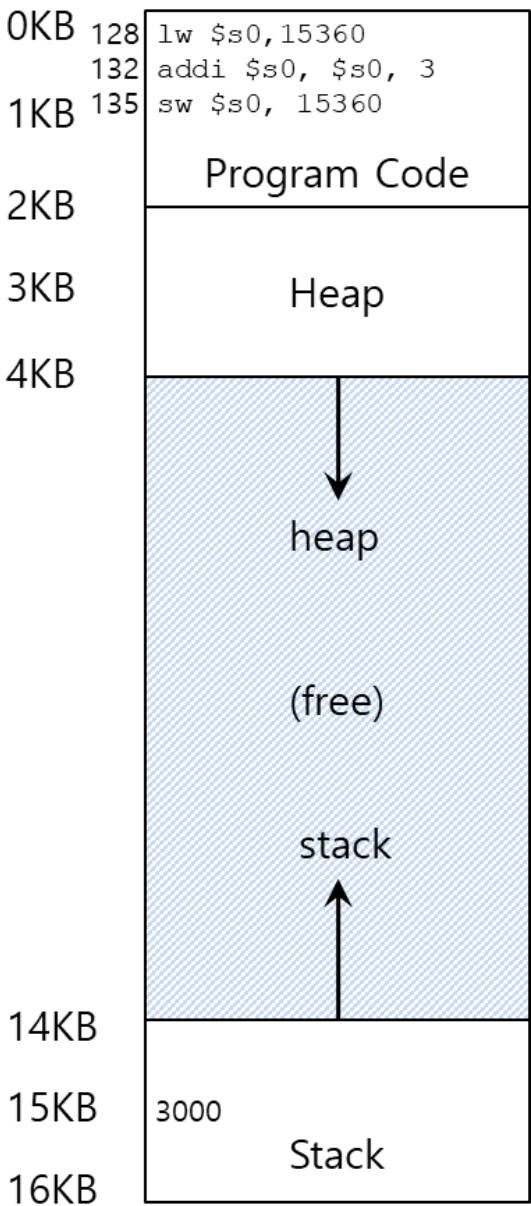
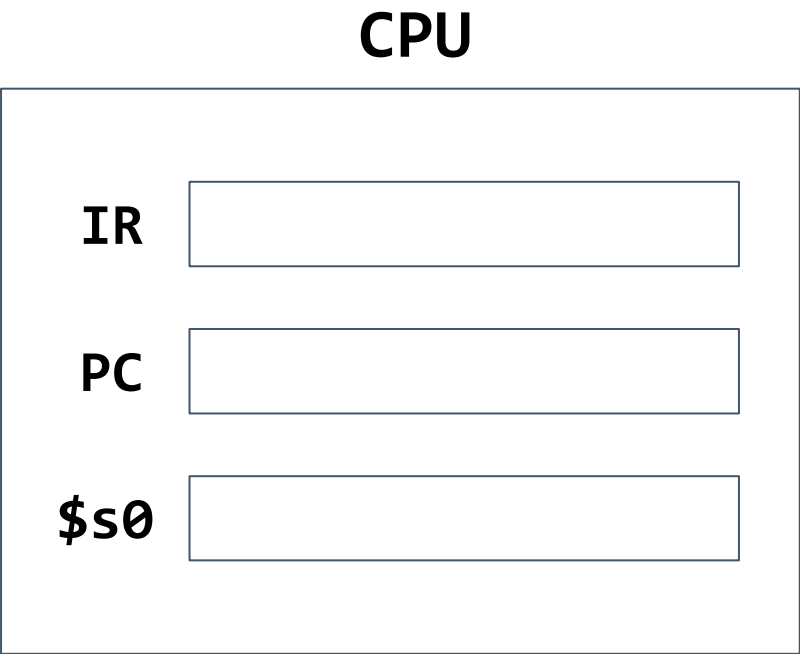
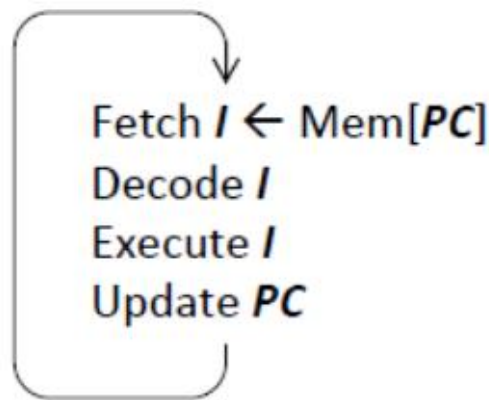
## Ejemplo

### Ensamblador

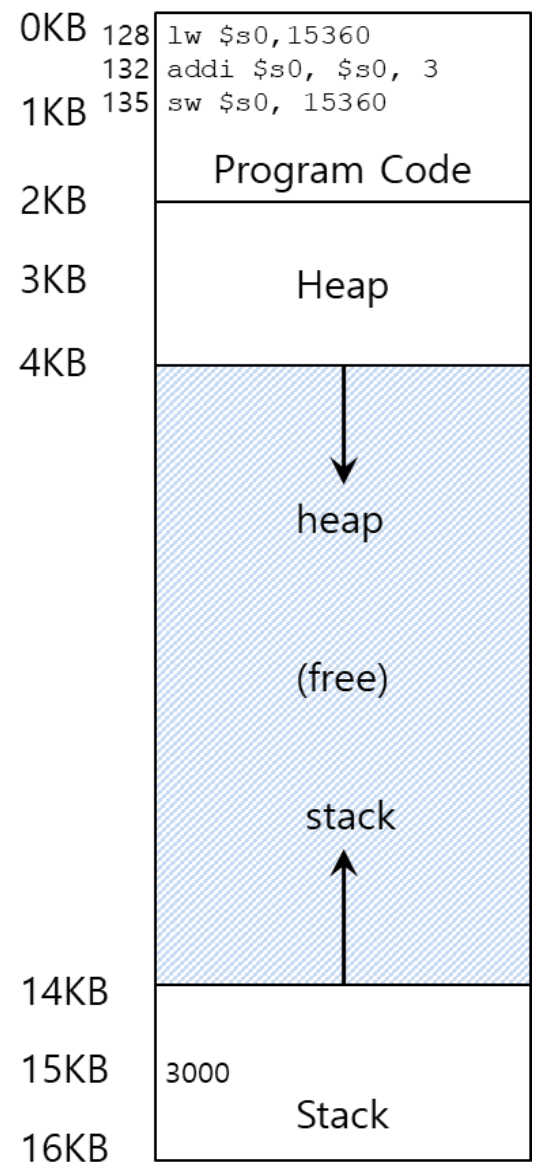
```
128 lw $S0, 15360  
132 addi $S0, $S0, 3  
135 sw $S0, 15360
```



# Contextualización



# Contextualización



128 lw \$S0, 15360

132 addi \$S0, \$S0, 3

135 sw \$S0, 15360

- Trae (fetch) instrucción desde dir. 128
- Ejecuta esta instrucción [Acceso a Mem]
  - Carga (load) dato desde dir. 15K

128 lw \$S0, 15360

132 addi \$S0, \$S0, 3

135 sw \$S0, 15360

- Trae (fetch) instrucción desde dir. 132
- Ejecuta esta instrucción [No acceso a Mem]
  - Incremento de \$s0 en 3

128 lw \$S0, 15360

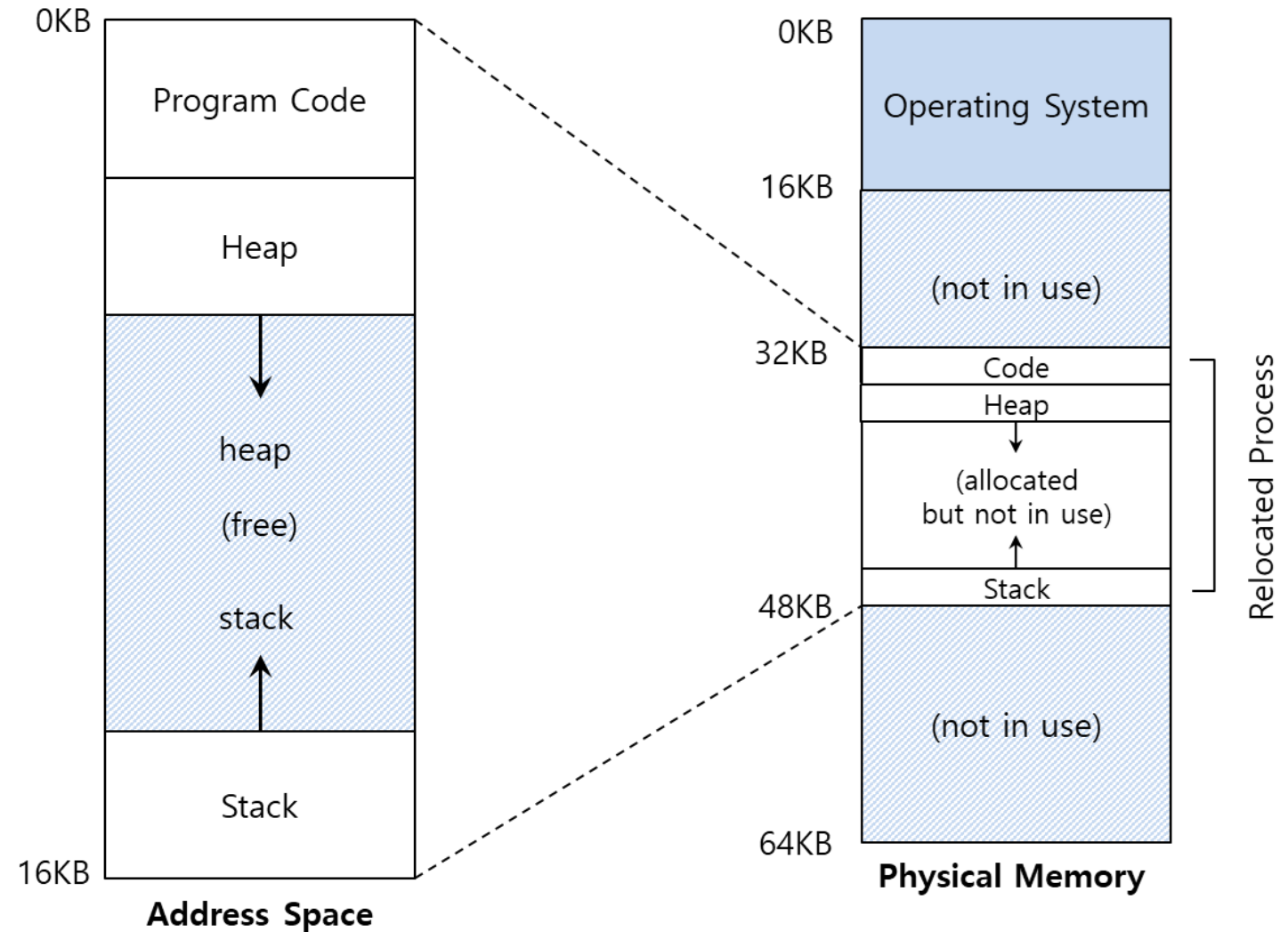
132 addi \$S0, \$S0, 3

135 sw \$S0, 15360

- Trae (fetch) instrucción desde dir. 135
- Ejecuta esta instrucción [Acceso a Mem]
  - Guarda (store) dato desde dir. 15K

# Reasignación del espacio de direccionamiento

- El SO ubica el proceso en una dirección diferente de 0.
  - Potencialmente **cualquier lugar!**



# Contextualización – Pregunta clave

¿Cómo virtualizar memoria de manera eficiente y flexible?



Virtual Address  
**VA**

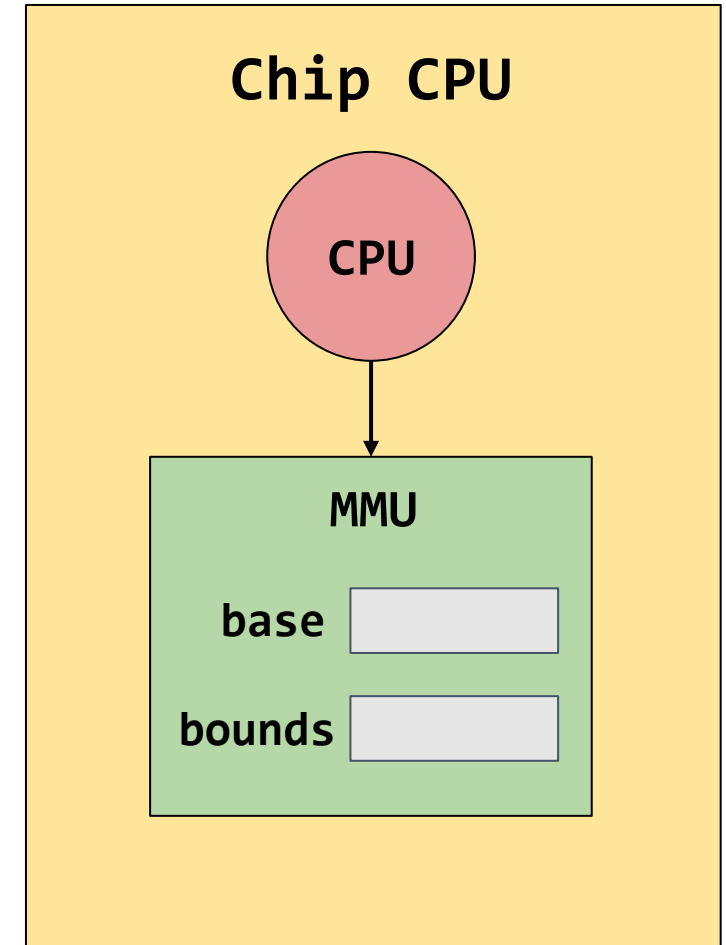
**Address  
Translation**

Physical Address  
**PA**

# Dynamic Relocation - Reasignación dinámica

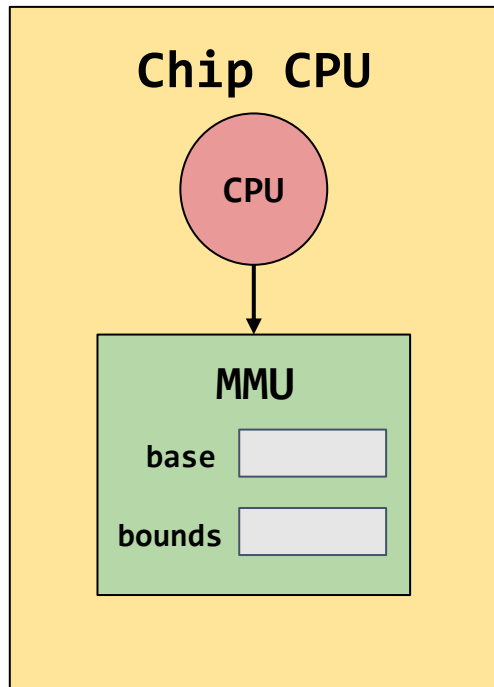
## Conceptos

- **Objetivo:** Traducir una dirección virtual a una física al agregar un **offset** fijo cada vez
- También conocida **Hardware-based reallocation** ó **base & bound reallocation**
- Esta técnica hace uso de los registros **base** y **bounds**:
  - **base:** Almacena la dirección física más pequeña.
  - **bounds:** Almacena el tamaño del segmento de memoria virtual asociada al proceso.

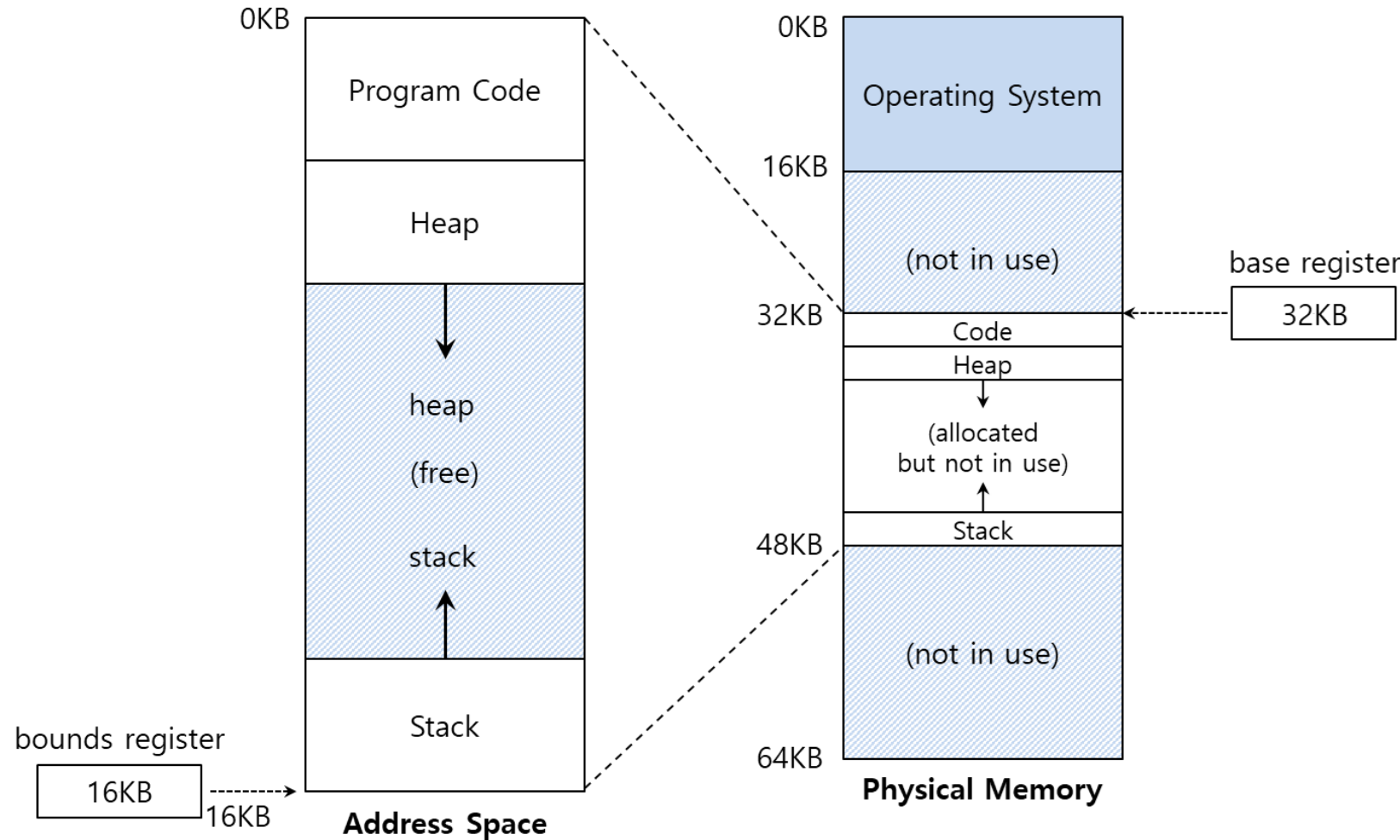


# Dynamic Relocation - Reasignación dinámica

## MMU (Memory Management Unit)



**MMU:** Hardware encargado de realizar la traducción de direcciones lógicas a físicas.



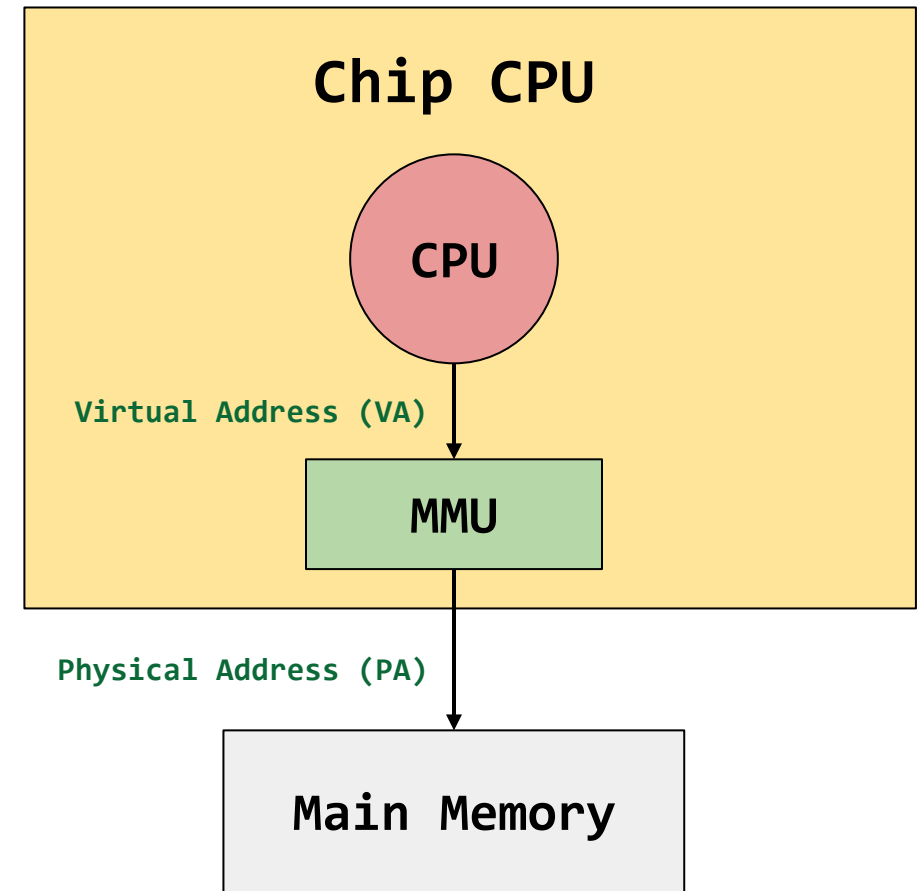
# Traducción de direcciones

- Cuando el proceso es creado, el SO decide **dónde** (en memoria física) será **cargada** su imagen de memoria.
  - Fija un valor para el registro base.
  - Las direcciones virtuales no deben ser mayores al valor en el registro bounds (limit) ni negativas.
- La traducción implica 2 pasos:
  1. Verificar que la dirección virtual sea valida (este dentro de los limites):

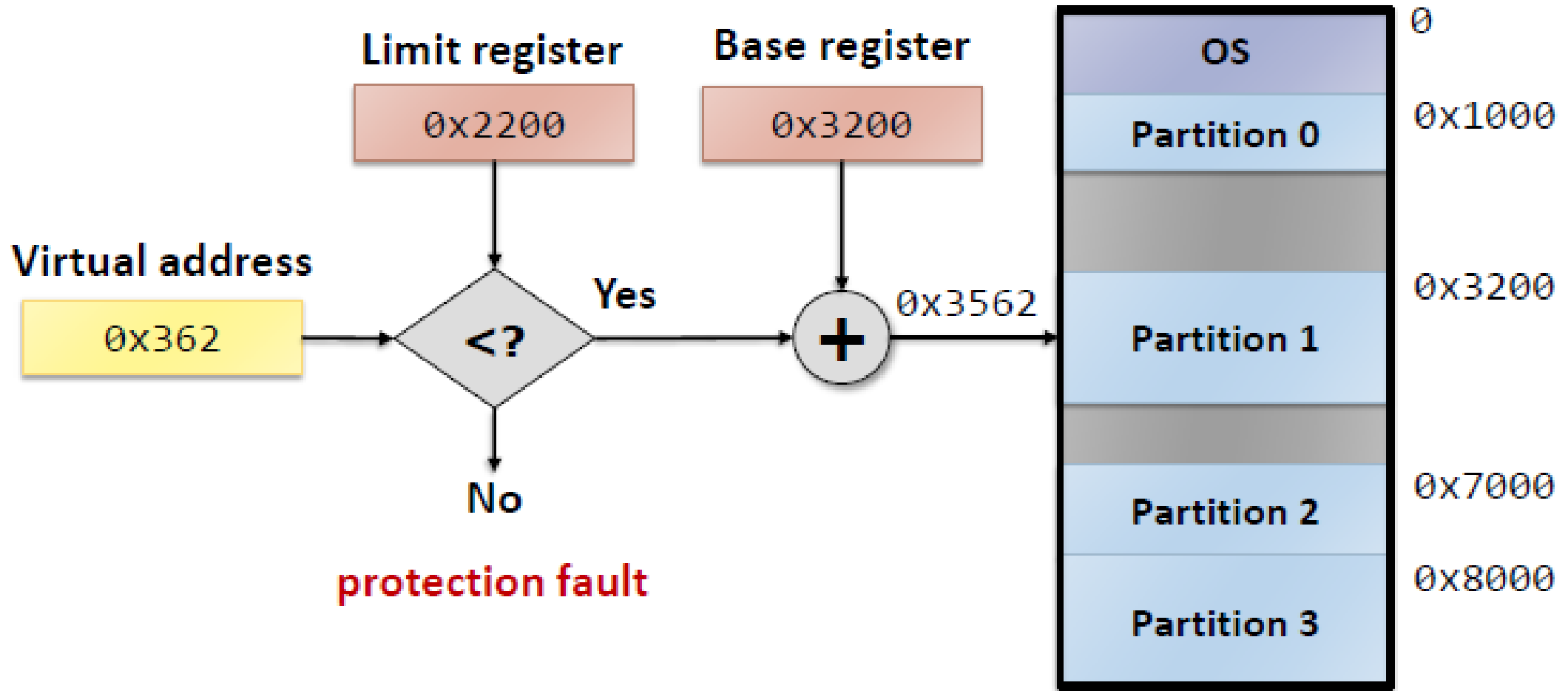
$$0 \leq VA < bounds$$

2. Realizar la traducción a la dirección física (PA) si la dirección virtual es valida (VA):

$$PA = VA + base$$



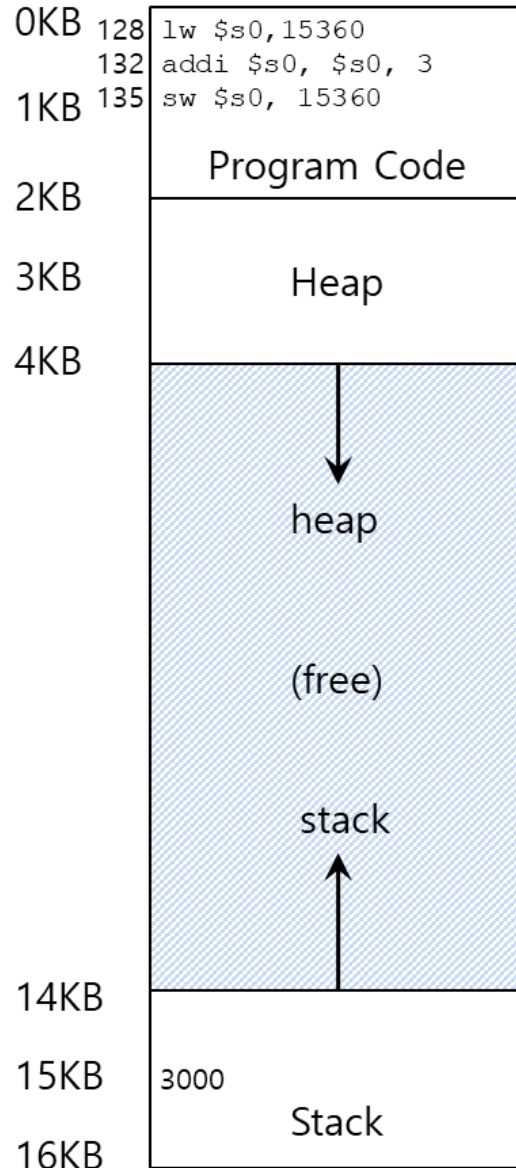
# Traducción de direcciones - Formulas



$$0 \leq VA < \textit{bounds}$$

$$PA = VA + \textit{base}$$

# Traducción de direcciones



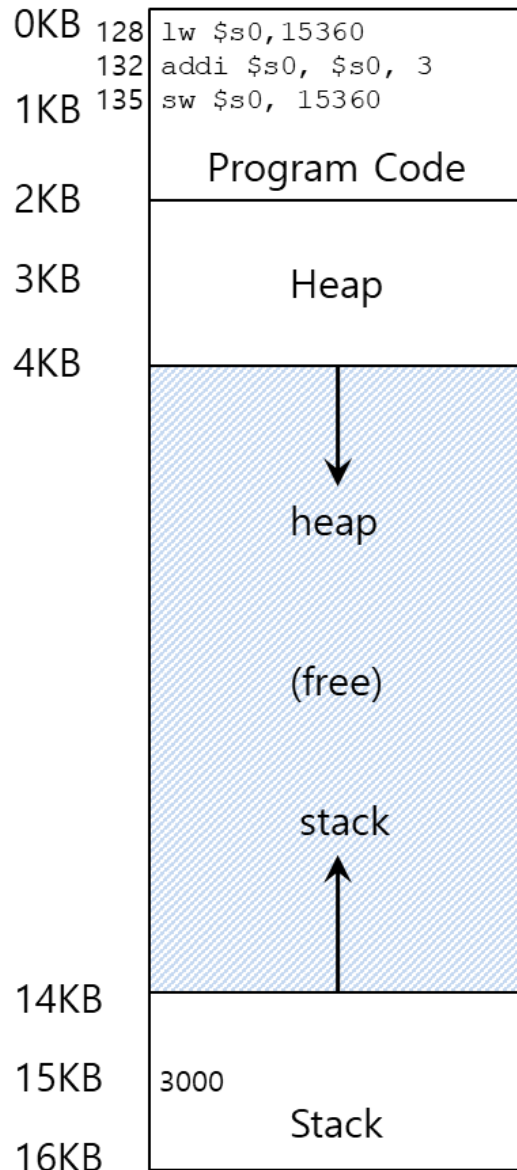
## Ejemplo - ¿Como un programa accede a memoria?

Dado el siguiente espacio de memoria asociado a un proceso. Analice qué sucede cuando se ejecuta la instrucción.

128 `lw $S0, 15360`

Asuma que el registro base tiene un valor de 32 K.

# Traducción de direcciones



Ejemplo - ¿Como un programa accede a memoria?

128 `lw $s0, 15360`

Para ejecutar la instrucción, se ejecutan dos accesos a memoria.

1. Traer la instrucción de la dirección 128

`I = Fetch[128]`

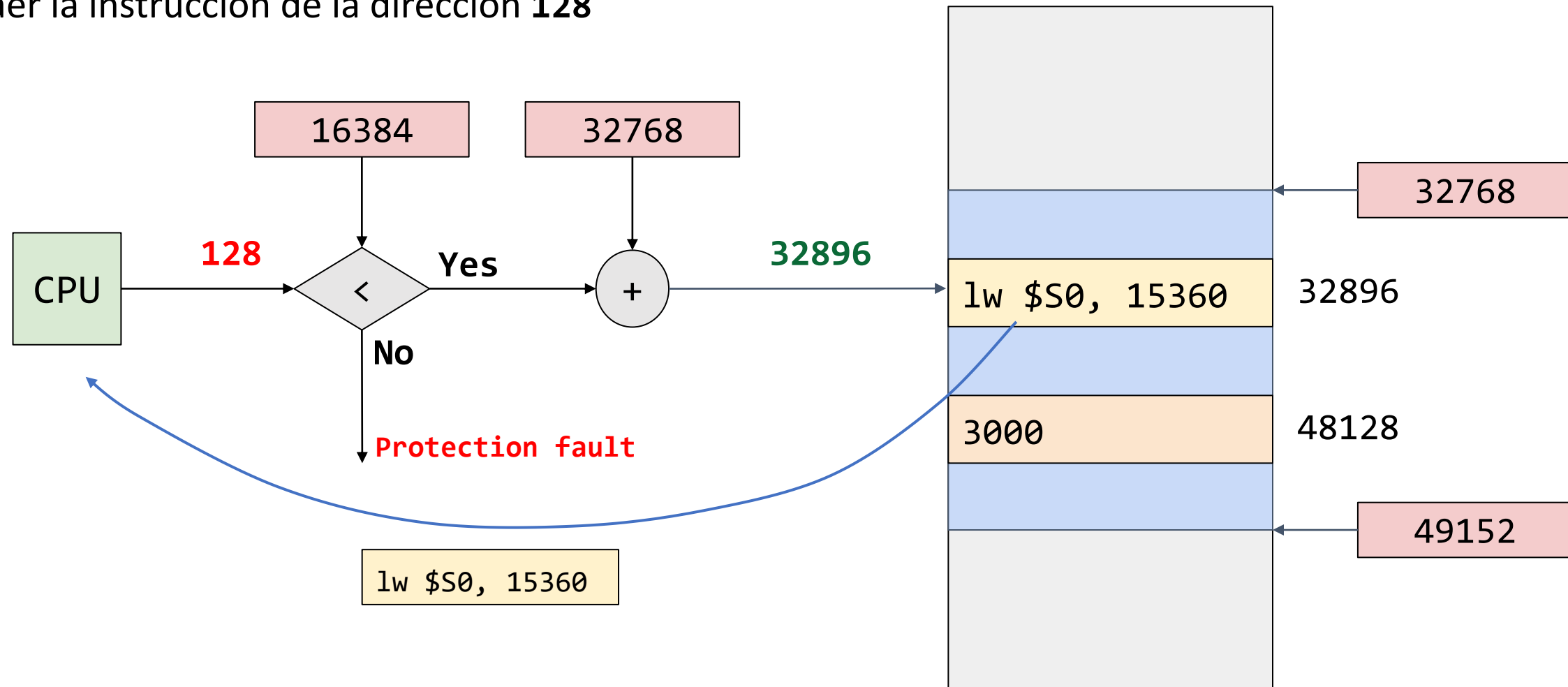
2. Ejecutar la instrucción I: `lw $s0, 15360`

`execute(I)`

# Traducción de direcciones

Ejemplo - ¿Como un programa accede a memoria?

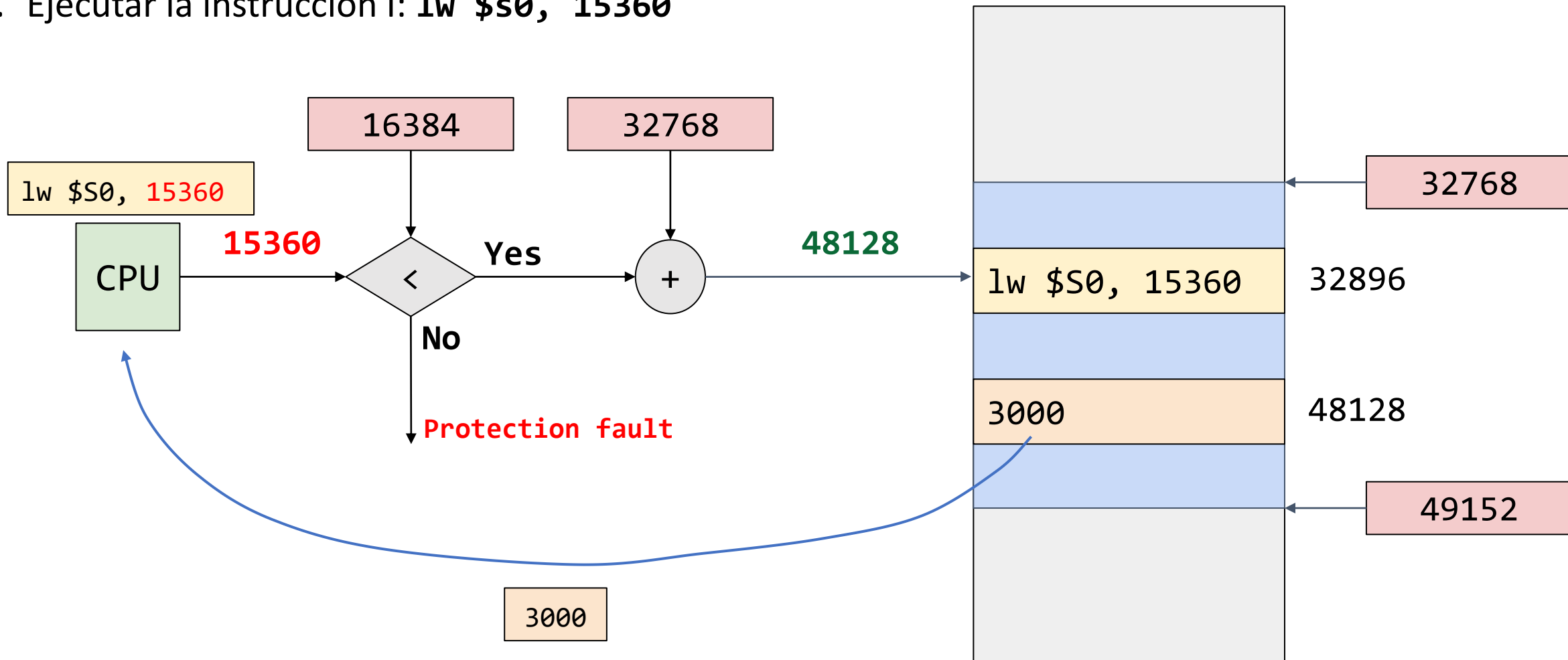
1. Traer la instrucción de la dirección 128



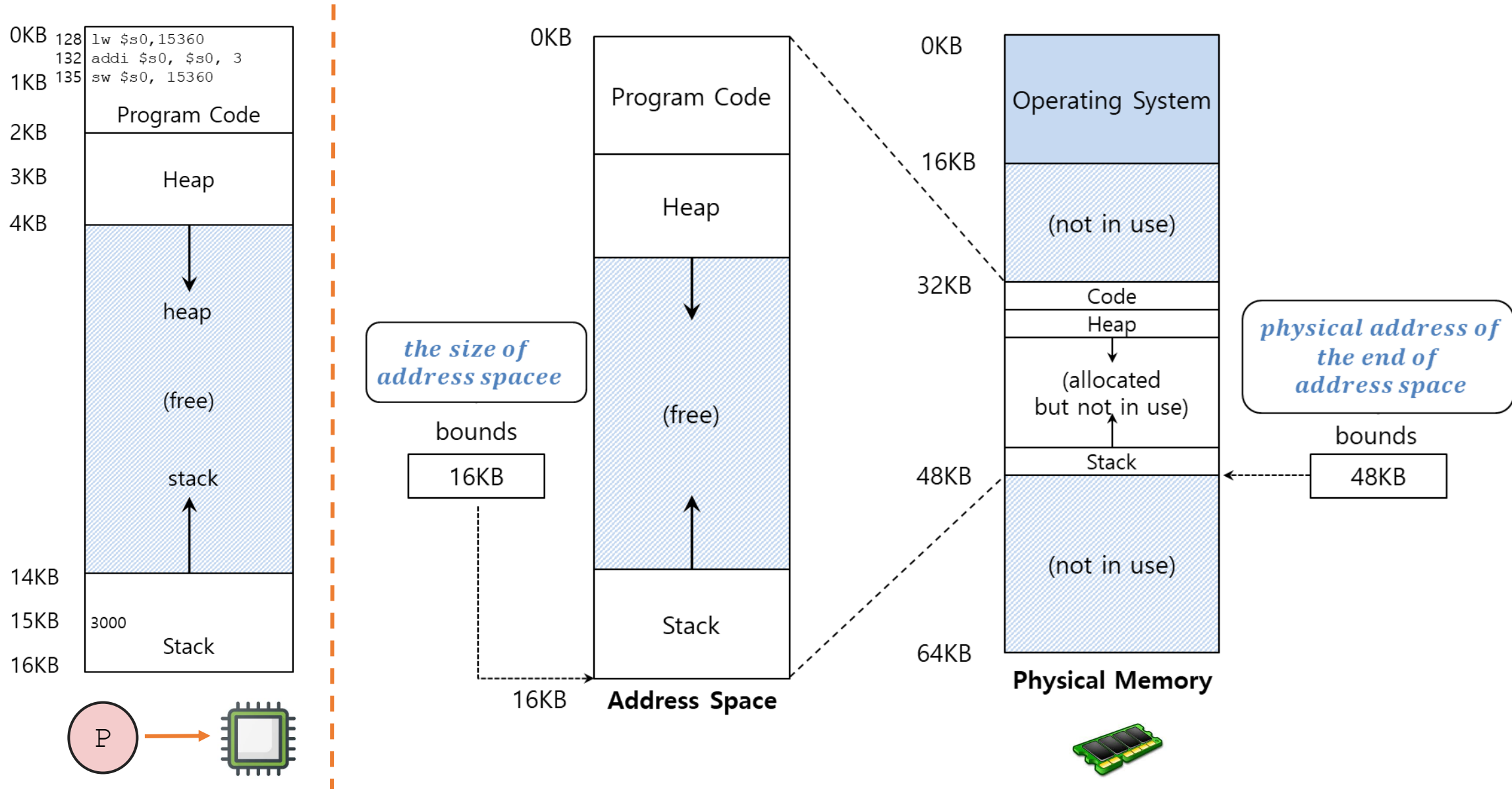
# Traducción de direcciones

Ejemplo - ¿Como un programa accede a memoria?

2. Ejecutar la instrucción l: `lw $s0, 15360`



# Traducción de direcciones



# Traducción de direcciones

## Ejemplo:

Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física de 64K a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

| Virtual Address (VA) | Physical Address (PA) |
|----------------------|-----------------------|
| 0                    |                       |
| 1 KB                 |                       |
| 3300                 |                       |
| 4400                 |                       |

# Traducción de direcciones

**Solución:** Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física de 64k a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

| Virtual<br>Address (VA) | Physical<br>Address (PA) |
|-------------------------|--------------------------|
| 0                       |                          |
| 1 KB                    |                          |
| 3300                    |                          |
| 4400                    |                          |

# Traducción de direcciones

**Solución:** Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física de 64k a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

| Virtual<br>Address (VA) | Physical<br>Address (PA) |
|-------------------------|--------------------------|
| 0                       |                          |
| 1 KB                    |                          |
| 3300                    |                          |
| 4400                    |                          |

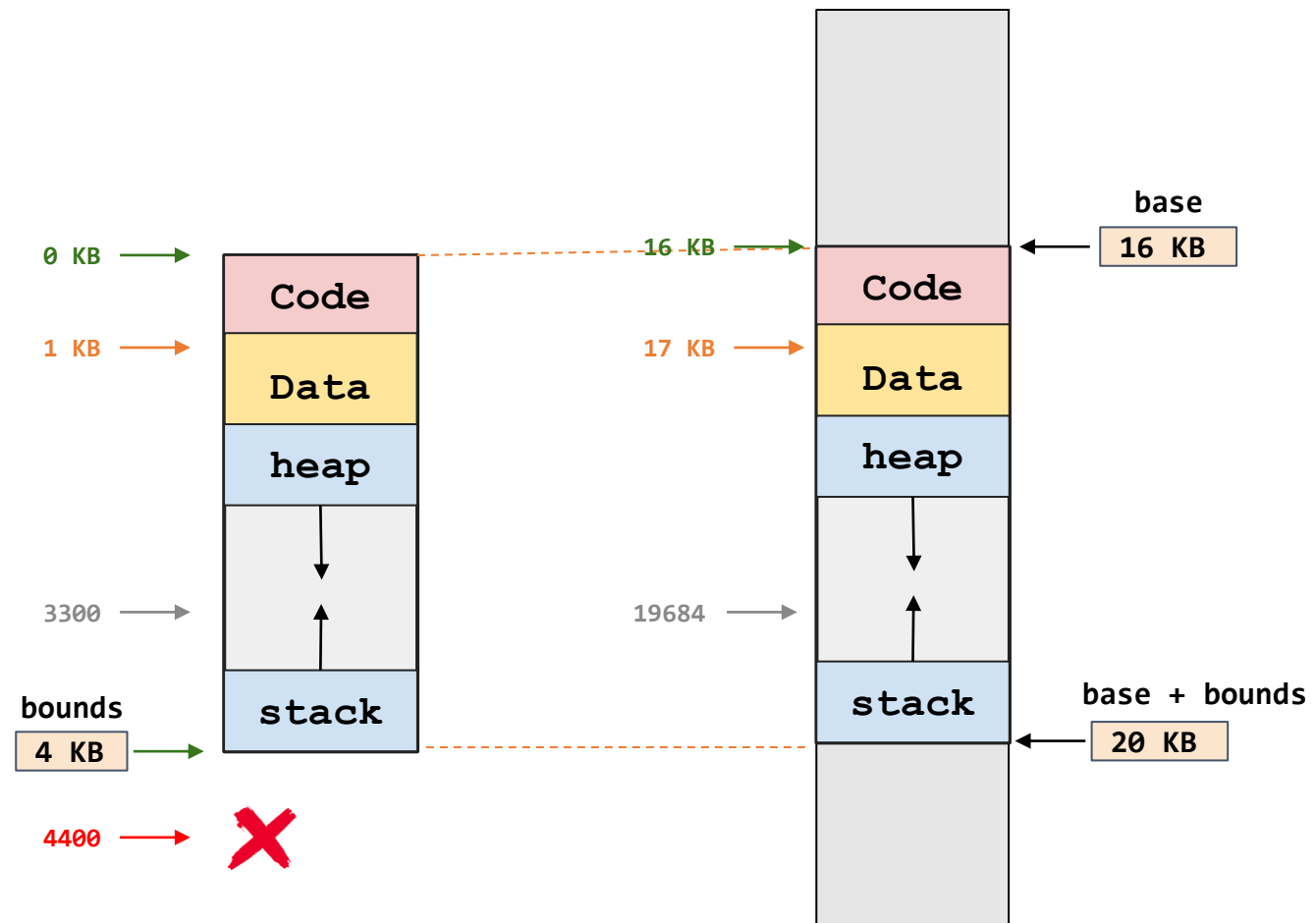
# Traducción de direcciones

## Ejemplo - Respuestas:

Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física de 64k a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

- **bounds = 4K**
- **base = 16k**
- **size(AS) = 4K**
- **size(PM) = 64 K**

| Virtual Address (VA) | Physical Address (PA)        |
|----------------------|------------------------------|
| 0                    | 16 KB                        |
| 1 KB                 | 17 KB                        |
| 3300                 | 19684                        |
| 4400                 | <b>Fault (Out of bounds)</b> |



# Traducción de direcciones

**Simulador:** Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física 64k a partir de 16KB.

```
./relocation.py -a 4k -p 64k -b 16k -l 4k
```

Address Space

|        |     |
|--------|-----|
| -----  | 0KB |
| Code   |     |
| -----  | 1KB |
| Stack  |     |
| -----  | 2KB |
| Heap   |     |
|        |     |
| v      |     |
| -----  | 3KB |
| (free) |     |
| ...    |     |
| -----  | 4KB |

<-- asize (-a 4k) = bounds/limit (-l 4k)

# Traducción de direcciones

**Simulador:** Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física 64k a partir de 16KB.

```
./relocation.py -a 8k -p 64k -b 16k -l 4k -n 4 -s 7
```

```
Address Space
----- 0KB
|   Code   |
----- 1KB
|   Stack   |
----- 2KB
|   Heap    |
|   |       |
|   v       |
----- 3KB
| (free)    |
|   ...     |
----- 4KB <-- bounds/limit (-l 4k) [proceso válido termina aquí]
#####
# fuera de #
# bounds   #
# (VIOLACIÓN)#
#####
----- 8KB <-- asize (-a 8k) [rango del generador aleatorio]
```

# Traducción de direcciones

**Simulador:** Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física 64k a partir de 16KB.

```
./relocation.py -a 8k -p 64k -b 16k -l 4k -n 4 -s 7 -c
```

```
ARG seed 7
```

```
ARG address space size 8k
```

```
ARG phys mem size 64k
```

```
Base-and-Bounds register information:
```

```
Base   : 0x00004000 (decimal 16384)
```

```
Limit  : 4096
```

```
Virtual Address Trace
```

```
VA 0: 0x00000a5c (decimal: 2652) --> VALID: 0x00004a5c (decimal: 19036)
```

```
VA 1: 0x000004d3 (decimal: 1235) --> VALID: 0x000044d3 (decimal: 17619)
```

```
VA 2: 0x000014d4 (decimal: 5332) --> SEGMENTATION VIOLATION
```

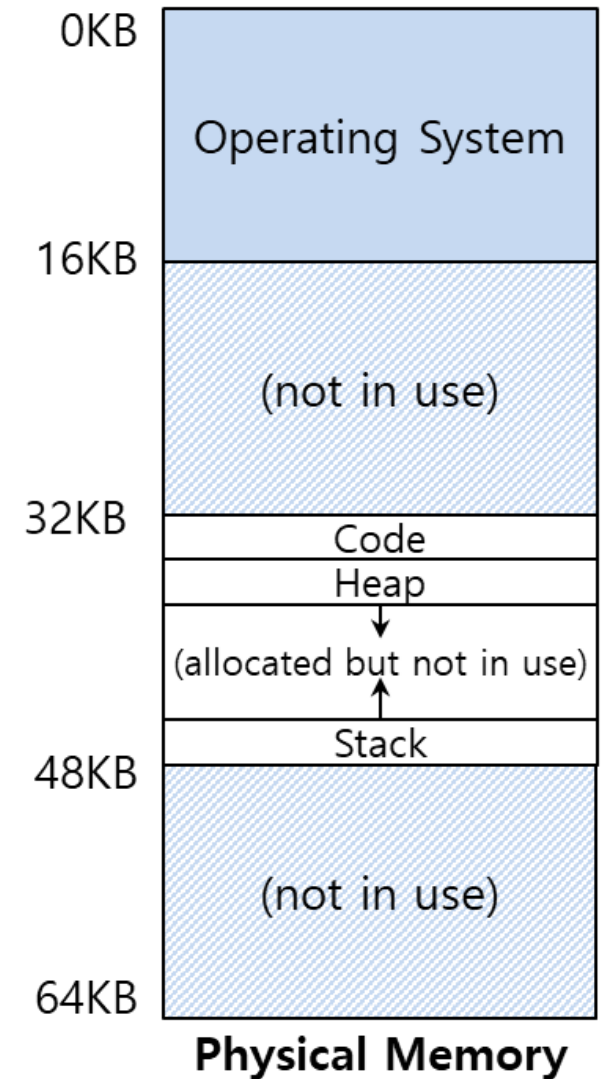
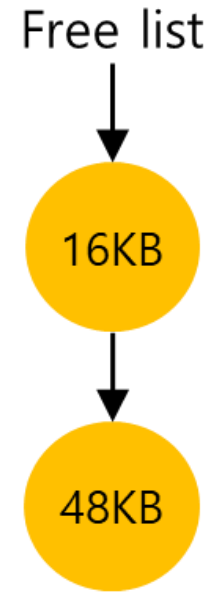
```
VA 3: 0x00000251 (decimal: 593) --> VALID: 0x00004251 (decimal: 16977)
```

# Acciones del sistema operativo

El SO debe realizar acciones para el correcto funcionamiento del sistema **base & bound**.

1. Cuando un proceso inicia su ejecución
2. Cuando un proceso termina
3. Cuando ocurre un cambio de contexto

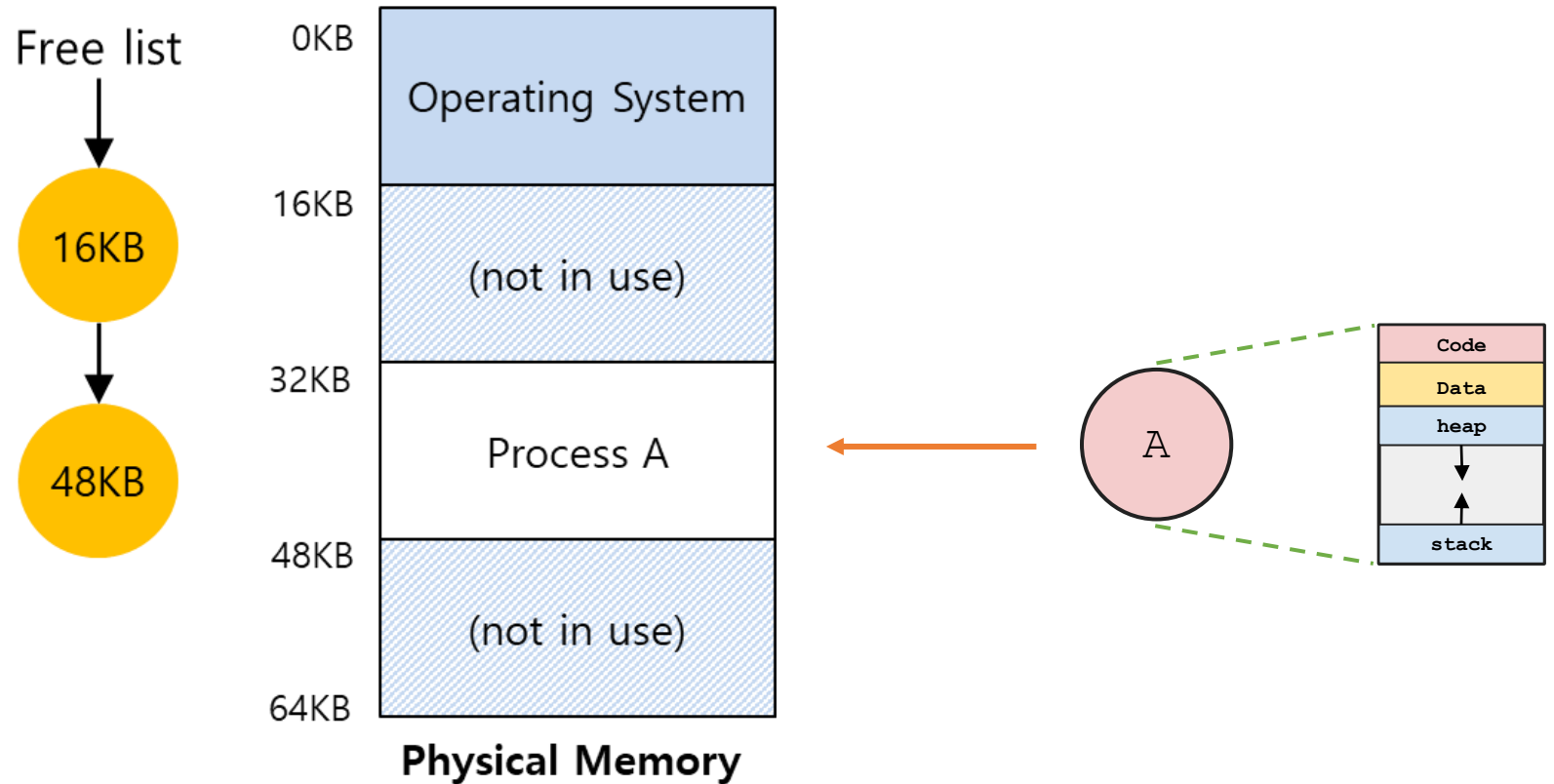
**Free list:** Lista de rangos de espacio libre en memoria física.



# Acciones del sistema operativo

## Creación de un proceso

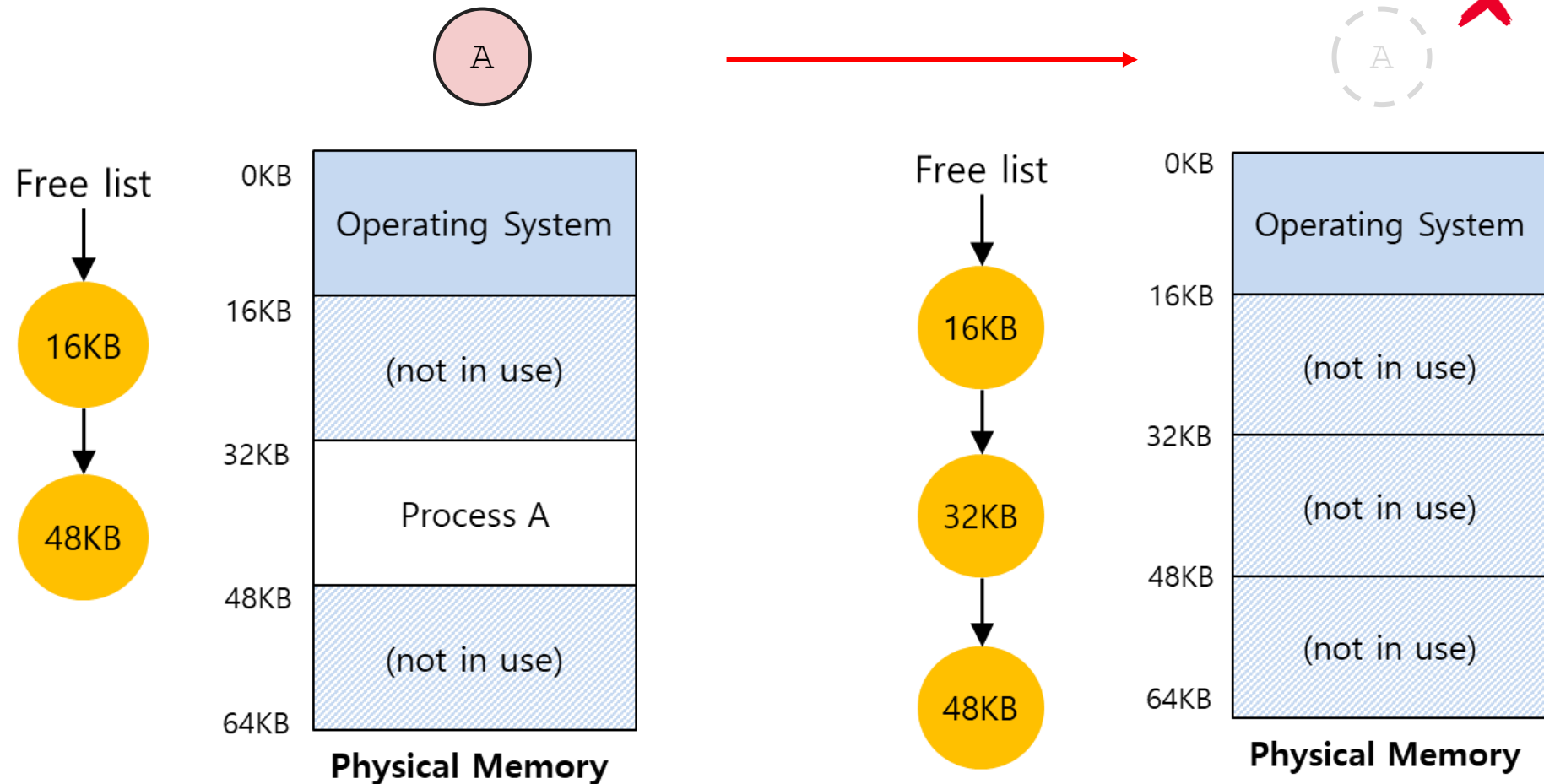
El SO debe encontrar un espacio para la memoria del nuevo proceso (**address space**).



# Acciones del sistema operativo

## Terminación de un proceso

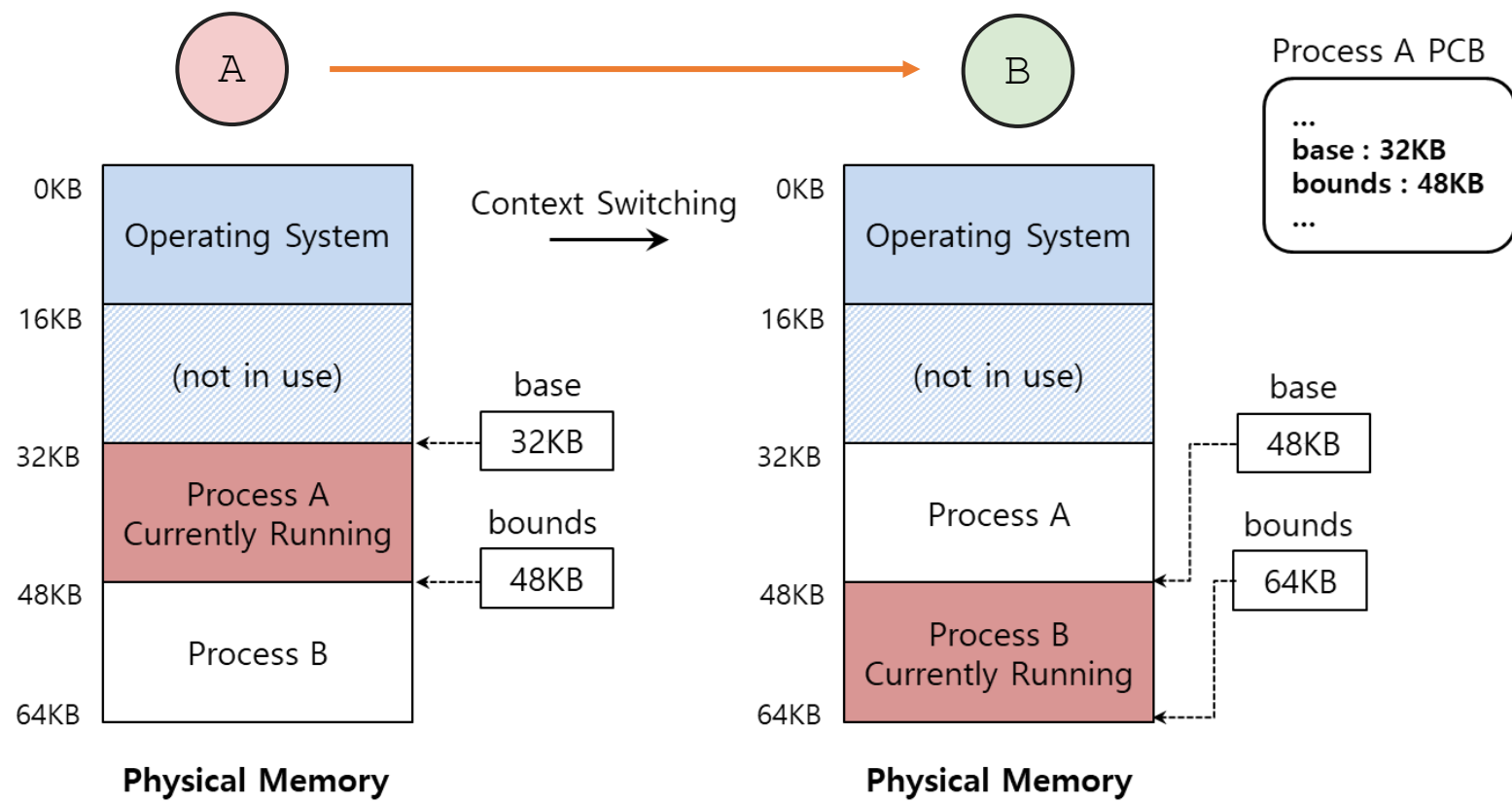
El SO recupera la memoria para su re-uso



# Acciones del sistema operativo

## Cambio de contexto

El SO almacena y restaura los registros **base** y **bound** en el PCB.



# Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [15](#))
- Videos de youtube: Clase 6 – Virtualización de memoria ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://web.stanford.edu/~ouster/cgi-bin/cs140-winter13/index.php>
- <https://bob.cs.ucdavis.edu/classes/s24-ecs150/index.html>